Test Project Outline – Module D – SwapLoop Rider SPA

Competition time

Competitors will have **3 hours** to complete this module.

Introduction

SwapLoop is a fictional Shanghai community pilot that offers safer alternatives to charging e-bike batteries indoors. Compatible delivery and private e-bikes exchange removable batteries at swap stations; e-bikes with integrated batteries use monitored **E-bike Charging Bays**.

This module builds the **mobile-first rider SPA** that replaces a native mobile client. The SPA consumes the provided **Module C Main Backend** REST API. Business rules, eligibility, reservation integrity, last-charge quarantine, charging simulation, and price snapshots remain authoritative in Module C. Module D owns presentation, client state, validation, loading and error handling, navigation, and QR-emulator integration.

General Description of Project and Tasks

Implement an independently runnable **single-page application (SPA)** that lets riders register, sign in, find stations, reserve services, complete swap or charging journeys, and view pay-as-you-go receipts.

High-level capabilities (details in Requirements):

- **Auth and profile:** register (two steps: vehicle profile + simulated Alipay link), login with bearer token, view and edit profile
- **Stations:** list, filter (type / nearby / compatible availability), station detail hub, reserve swap or charging hold
- **Station QR:** integrate the provided QR emulator web component; parse station deep links; open the station hub
- **Activity:** active service lifecycle (countdown, start / confirm / cancel for swap; start / live poll / collect / cancel for charging) and recent services with receipts
- **Safety UX:** handle Module C last-charge `409` without inventing availability or quarantine rules in the client
- **Errors and accessibility:** distinct handling for common HTTP statuses; keyboard-operable flows; colour-independent status

Environment and stack

- Build a **JavaScript SPA** with a modern framework (e.g. React, Vue, or Angular) and **client-side routing**. Reloading a deep-linked URL must restore the same view after auth state is read from storage (except unsaved form input).
- Consume the **Module C Main Backend** under `/api/v1`. Paths, query parameters, and schemas are in `assets/api/main-backend.openapi.yaml` (offline Swagger UI: `assets/api/main-backend-docs/index.html`). Assessment and local hosts are in Hosts, seed accounts, and fixtures.
- Persist the opaque bearer token in `localStorage` (or equivalent) and send `Authorization: Bearer <token>` on protected calls.
- Embed the competition **QR emulator** from `assets/qr-code-emulator/`. Integration steps are in Station QR scan and `assets/handouts/handout-qr-emulator-integration.md`.
- Prefer native `fetch` (or the framework's HTTP client). Display only prices, quarantine outcomes, and availability the Main Backend returned.

Hosts, seed accounts, and fixtures

Replace `cXX` / `YYYY` with the competition username and PIN. Local URLs are for development.

Service	Assessment	Local
Main Backend (`/api/v1`)	`https://cXX-YYYY-module-c.sitc.skillsit.eu/api/v1`	typically `http://localhost:5000/api/v1`
Station Service (QR emulator `service-url` only)	`https://cXX-YYYY-station-service.sitc.skillsit.eu`	`http://localhost:4020`
QR tester (choose the active poster)	`https://cXX-YYYY-station-service.sitc.skillsit.eu/qr-code-emulator`	`http://localhost:4020/qr-code-emulator`

The SPA must not call Station Service except through `<swaploop-qr-emulator>`. Last-charge telemetry and live charging sessions are proxied by the Main Backend.

Plaintext password for all seeded users: `password123`. Reset the Main Backend seed between marking aspects when a 10-second hold has expired.

Email	Status	Profile	Use for
`lin.xiaoyu@swaploop.test`	ACTIVE	SWAPPABLE / SL-48	Login, swap journey, 409 at station-005
`chen.wei@swaploop.test`	ACTIVE	INTEGRATED / GB-AC-48	Charging journey
`zhao.min@swaploop.test`	ACTIVE	SWAPPABLE / SL-48	Spare swappable rider
`sun.hao@swaploop.test`	SUSPENDED	INTEGRATED / GB-AC-60	Login `403`

Last-charge 409 (swap): as `lin.xiaoyu@swaploop.test`, reserve a SWAP service at **station-005**. The only READY SL-48 pack there is `battery-007` (sustained-heat fixture). The Main Backend should refuse with `409 CONFLICT`. Do not use station-002 for this test: it also has another READY SL-48 pack that may still reserve after the spike pack is skipped.

Technical constraints

- Display API timestamps as returned (ISO 8601 with offset). Do not invent or recompute times client-side.
- For the **nearby** station filter, use the browser Geolocation API and pass `lat`, `lng`, and `radiusMeters` (default **1500**) on `GET /stations` as in the OpenAPI. During development and assessment, set the browser location to **Shanghai** in DevTools (Sensors / Location), because the seeded stations are in Shanghai.
- Real Alipay Open Platform integration, real payments, real hardware, OAuth / token refresh / email verification / password reset are **out of scope**.
- Only **pay-as-you-go** payment is in scope. Other payment methods (for example monthly plans) will be provided later.

Physical vocabulary

Use this vocabulary consistently in the UI:

Term	Meaning
SwapLoop Station	Full service location (`SWAP` , `CHARGING` , or `HYBRID`). **Only stations have a poster QR.**
Battery Swap Cabinet	Equipment that stores and charges swappable batteries (no QR in this competition)
Battery Slot (Swap Bay)	One compartment that holds at most one battery (API unit often `SWAP_BAY`)
E-bike Charging Bay (Bike Bay)	Bay that charges a whole e-bike with an integrated battery (`BIKE_BAY`)

Compatibility catalog (do not invent parallel codes):

- Swappable: `batteryMode` `SWAPPABLE` + `batteryType` `SL-48` | `SL-60` (derived `voltageClass` `48V` / `60V`)
- Integrated: `batteryMode` `INTEGRATED` + `connectorType` `GB-AC-48` | `GB-AC-60`

Roles in scope

Role	Module D behaviour
Registered rider (swappable)	Register/login, reserve swap, start + confirm, receipt
Registered rider (integrated)	Register/login, reserve charging bay, start + poll + collect, receipt
Unauthenticated visitor	Login and registration only; no station list, station detail, scan, or Activity until signed in

Assets

```
assets/
  api/
    main-backend.openapi.yaml          # Main Backend contract (/api/v1)
    main-backend-docs/                 # Offline Swagger UI
  handouts/
    handout-qr-emulator-integration.md # embed <swaploop-qr-emulator>
  qr-code-emulator/
    swaploop-qr-emulator.js            # IIFE bundle (registers the custom element)
    README.md
  wireframes/                          # page-structure guidelines (not pixel-perfect)
```

Wireframes in `assets/wireframes/` are **guidelines for page structure** only. Do not implement them pixel by pixel. Italic or instructional copy on the wireframes is author guidance for competitors — it must **not** appear in the rider UI. Visible labels, sample names, and placeholder text may be replaced as long as the required behaviours remain.

Requirements

The SwapLoop rider SPA shall implement the behaviours below.

App chrome and navigation

The SPA should feel like a native rider app, not a desktop website with a top-only menu.

Once the rider is signed in, keep a **sticky navigation bar at the bottom of the viewport** on Stations, Scan, Activity, and Profile. It stays visible while the main content scrolls. Include at least these destinations, with labels and/or icons:

- **Stations** — station list
- **Scan** — station QR emulator
- **Activity** — active and recent services

- **Profile** — rider account and vehicle details

The current destination must be visually distinct. Tapping a destination must switch that view without a full page reload. Login and the two-step registration screens do not need the bottom bar.

Leave enough bottom padding so cards, reserve buttons, and Activity actions are not hidden behind the bar.

Responsive design and accessibility

Design for a phone in a rider's hand first: a single column, large enough tap targets, and a layout that still works if an assessor stretches the window to a desktop width. Do not depend on hover-only controls for reserve or Activity actions.

Keyboard users must be able to complete the service journey without a pointer: reserve, start, confirm, collect, and cancel must be reachable and operable with Tab and Enter/Space, with a visible focus state.

Status must not rely on colour alone. Pair colour with text and/or icons so a rider can tell a station is closed, a hold is expiring, or a request failed. Countdown expiry, major service-state changes, and API errors need a clear on-screen message, not only a colour shift.

Authentication and profile

Anyone who is not signed in must only see **login** and **registration**. Station list, station detail, Scan, and Activity stay behind authentication. `GET /stations` on the Main Backend is public, but this SPA must not show station, scan, or Activity views until the rider has a stored token.

Registration is a two-step flow. First collect email, password, display name, and a vehicle profile that depends on battery mode. For `SWAPPABLE` riders require `batteryType` `SL-48` or `SL-60` and omit `connectorType`. For `INTEGRATED` riders require `connectorType` `GB-AC-48` or `GB-AC-60` and omit `batteryType`. Do not send `voltageClass` on register; Module C derives it and returns it on the rider profile.

The second step is a **simulated Alipay link** for pay-as-you-go. Show a mock Alipay QR / linking UI, wait a short delay, then show success and enable **Create account**. Do not call a live payment API.

On successful register, store the returned token and enter the authenticated app. Returning riders sign in with email and password. If login is rejected with `403` (suspended account), stay on the login screen and show that message — do not store a token.

The profile screen loads the current rider and displays display name, email, role, status, battery mode, battery type **or** connector type (whichever applies), the derived voltage class, and `currentBatteryId` when a swappable rider currently holds a pack. Riders may edit display name and vehicle profile, using the same mode-driven field rules as registration. Sign out must clear the stored token and return to login.

Stations list and filters

Show a scrollable list of SwapLoop stations. Load availability that matches the signed-in rider's vehicle profile so each card can say whether a ready battery (swap) or charging bay (integrated) is actually usable for them. This list is only for authenticated riders.

Riders must be able to narrow the list. Filter by station type: all stations, or only `SWAP`, `CHARGING`, or `HYBRID` (`type` on `GET /stations`). Offer a **compatible availability** filter that keeps stations with a ready pack or bay for the signed-in profile: `service=SWAP` with `batteryType`, or `service=BIKE_BAY` with `connectorType`. Offer a **nearby** filter using the browser Geolocation API with `lat`, `lng`, and `radiusMeters` (default **1500**, about 1.5 km). During assessment the browser location is set to Shanghai.

Each station card shows name, type, lifecycle status, and address or hours when the API provides them. With nearby on, show distance (`distanceMeters`). Make it obvious whether a compatible battery or charging bay is ready for this rider — not only that the station exists. Closed or suspended stations that still appear in the list must be recognisable from lifecycle status (text and/or icon, not colour alone).

Empty **filtered** lists must not all look the same. Distinguish "nothing compatible is ready now" from "no station is nearby". Opening a card goes to that station's detail page.

Station detail hub

This screen is the on-site hub for one station. Include the signed-in rider's availability so the page can say whether a compatible battery or charging bay is ready. Show identity, address, status, hours, compatibility (services, battery types, connector types, voltage classes), and short guidance for this rider.

If the rider is eligible and the station is `ACTIVE`, offer **Reserve battery** for swappable riders or **Reserve charging bay** for integrated riders. Creating a hold asks the main backend for a `SWAP` or `CHARGING` service at this station. The hold length is the API `expiresAt` value — **10 seconds** in the competition API so expiry is easy to mark (a production hold would typically be 15–30 minutes). On success, show the active hold and a clear path to **Activity**. Start or cancel from Activity before that clock runs out; reset the Main Backend seed if you need a fresh hold.

If the rider already has an **active service at this station**, show that hold or in-progress state and a primary control to open Activity. Do not show a misleading "No ready battery / bay" chip while that hold is still theirs. If they already have an **active service at another station**, block a new reserve here and link to Activity instead.

Last-charge safety (swap): The main backend may refuse a swap hold with `409 CONFLICT` after quarantine or missing telemetry. Show rider-facing copy such as "The selected battery is not available anymore". Reload this station so availability is current, then hide or disable reserve when nothing compatible remains. Do not re-implement spike or sustained-heat rules in the client, and do not silently send the rider to another station. Use the **station-005** fixture in Hosts, seed accounts, and fixtures.

Unknown station ids — including a bad QR parse that still opened this hub — must surface a `404` in rider-facing language.

Station QR scan

Give riders a **Scan** screen that embeds the provided `<swaploop-qr-emulator>` web component. Do not use a device camera. The emulator supplies the poster QR that would be printed at a SwapLoop Station. The SPA must **not** call Station Service for last-charge telemetry or live bike-bay charging sessions — the Main Backend proxies those. The only Station Service traffic from the rider UI is through this component's `service-url`.

How to load the IIFE bundle, set `service-url`, start a scan by changing `scan-request-id`, and read `event.detail.payload` from the `qr-scan` event is documented in `assets/handouts/handout-qr-emulator-integration.md`, including React, Vue, and Angular examples. The script is `assets/qr-code-emulator/swaploop-qr-emulator.js`.

A **QR tester** is also provided so you can choose and check the active poster code without a camera. Open it at `https://cXX-YYYY-station-service.sitc.skillsit.eu/qr-code-emulator` (replace `cXX` / `YYYY` with your workstation host; local Station Service: `http://localhost:4020/qr-code-emulator`). Use it to set which station QR the emulator will return, then trigger a scan in the SPA.

Every station poster encodes the same kind of deep link:

```
https://app.swaploop.test/stations/{stationId}
```

When the component fires `qr-scan`, parse that string. Accept the full URL, and also a bare `station-…` id. Any other shape is a mismatch — show a clear rider-facing message and do not invent a station.

If parsing yields a station id, open that station's detail hub. Treat the scanned value as untrusted until the main backend successfully returns that station. A forged or unknown id must not look like a real reservation or a valid hub.

Activity hub

Activity is where a signed-in rider manages the service they currently hold and looks back at recent ones. Load the rider's **active** item and **recent** list from the main backend.

For an active **swap** (`SWAP`), while the hold is `RESERVED` show a countdown from `expiresAt` (same **10-second** competition hold as on the station hub) and offer **Start swapping** and **Cancel**. After start (`STARTED`), offer **Confirm swap** and **Cancel**. The physical handoff does not need another QR scan.

For an active **charging** session (`CHARGING`), while `RESERVED` show the same kind of countdown, **Start charging**, and **Cancel** (cancel only in `RESERVED`). Once charging has started, poll live status about once per second and show SOC, power, temperature, and any `endsAt` countdown. The competition charging simulation lasts about **15 seconds**. Stop polling when the main backend reports the session complete / `READY_FOR_COLLECTION`, then offer **Collect bike**. Do not put a **Mark ready** control on the happy path — the backend advances to ready through the status poll.

After confirm or collect, show a **receipt** from `priceYuan` / `priceCode` in CNY. Do not calculate or guess the price in the SPA. List recent finished services and let the rider reopen those receipt fields. If there is nothing active and nothing recent, say so clearly and link back to Stations.

Safety-stop and error states from the main backend (for example `SAFETY_CUTOFF`) must appear in plain rider-facing language.

Client state and API usage

Follow the OpenAPI for exact paths. Expected capability groups:

```
POST /auth/login, POST /auth/register
GET  /me, PATCH /me
GET  /me/activity
GET  /stations, GET /stations/{id}
POST /services, GET /services/{id}
POST /services/{id}/start
GET  /services/{id}/charging-status
POST /services/{id}/confirm
POST /services/{id}/collect
POST /services/{id}/cancel
```

Handle `401`, `403`, `404`, `409`, `422`, and `5xx` with actionable, accessible messages and no internal stack traces. A `401` on a protected rider action (missing or unknown token) must clear the stored session and return to login. A `403` on a later protected call (for example the account was suspended) must also clear the session and return to login. Login `403` for a suspended account stays on the login screen with the message, as specified under Authentication.

Prevent double-submit while a mutation is pending. Keep server state separate from transient UI state; show loading or stale indicators rather than presenting cached availability as current.

Assessment

Assessment is by expert judgement against the marking scheme in **Google Chrome**, using the Module C reference API (reset seed) and Station Service for the QR emulator. Automated tests may be used where provided; a manual walkthrough of register → find → reserve → Activity complete → receipt is required.

Competition holds last **10 seconds**: after Reserve, open Activity and Start immediately, or reset the Main Backend seed between aspects. Charging live telemetry runs for about **15 seconds** before `READY_FOR_COLLECTION`.

Mark distribution

The mark distribution for this project is as follows:

WSOS SECTION	Description	Points
1	Work organization and self-management	1
2	Communication and interpersonal skills	1.5
3	Design Implementation	2.5
4	Front-End Development	12
Total		17